We need a language mechanism for customization points

Document #:	P2279R0
Date:	2021-01-15
Project:	Programming Language C++
Audience:	EWG

Reply-to: Barry Revzin <barry.revzin@gmail.com>

Contents

1 Introduction
- 1.1 Polymorphism using `virtual` member functions
- 1.2 Parametric Polymorphism
- 1.3 Named Conformance vs Structural Conformance

2 Existing Static Polymorphism Strategies
- 2.1 Class Template Specialization
- 2.2 Pure ADL-based customization
- 2.3 Customization Point Objects
- 2.4 `tag_invoke`
- 2.5 Better Enough?
- 2.6 The swap example

3 Relevant Work
- 3.1 Customization Point Functions
- 3.2 Reflective Metaprogramming
- 3.3 C++0x Concepts
- 3.4 The contenders

4 References

§ 1 Introduction

C++ is a language that lauds itself on the ability to write good, efficient generic code. So it's a little strange that here we are in C++20 and yet have surprisingly little language support for proper customization.

It's worth elaborating a bit on what I mean by "proper customization." There are a few facilities that I think of when I say this (in no particular order):

1. The ability to see clearly, in code, what the interface is that can (or needs to) be customized.
2. The ability to provide default implementations that can be overridden, not just non-defaulted functions.
3. The ability to opt in *explicitly* to the interface.
4. The inability to *incorrectly* opt in to the interface (for instance, if the interface has a function that takes an `int`, you cannot opt in by accidentally taking an `unsigned int`).
5. The ability to easily invoke the customized implementation. Alternatively, the inability to accidentally invoke the base implementation.
6. The ability to easily verify that a type implements an interface.
7. The ability to present an atomic group of functionality that needs to be customized together (and diagnosed early).

This list is neither complete (I will add a few additional important requirements later in the paper) nor do I consider all of these aspects to be equally important, but it's a good list to introduce this discussion.

§ 1.1 Polymorphism using `virtual` member functions

C++ has precisely one language feature that meets all of these criteria: `virtual` member functions.

1. Given an interface, you can clearly see which functions are `virtual` (or pure `virtual`), with the caveat that in some cases these functions may be inherited ✔️.

2. You can have functions that are pure `virtual` (which must be overriden) alongside functions which are `virtual` but contain default implementations. This distinction is easy to understand and implement ✔️.

3. Implementing a `virtual` polymorphism-based interface can only be done via inheritance, which is explicit ✔️. Within that, each individual member function override can be marked `override`. This is not mandatory, but can be enforced with `-Wsuggest-override`, which makes overrides even more explicit ✔️ (and avoids accidental overrides).

4. If you attempt to override a function incorrectly, it's a compile error at point of definition ✔️ (as opposed to being an error at point of use or, worse, not an error at all):

```
struct B {
    virtual void f(int);
};

struct D : B {
    void f(unsigned int) override;  // error here
};
```

5. Given a pointer to the interface, just invoking the function you want will automatically do virtual dispatch per the language rules, which automatically invokes the most derived implementation. This requires no additional work on the part of either the interface author or interface user ✔️.

6. Checking if a type `T` implements an interace `I` is as easy as checking if `derived_from<T, I>` holds ✔️.

7. If there is an interface has two pure `virtual` member functions, there cannot be an implementation of that interface that only implements one of them. You must implement both, otherwise you cannot even construct an instance of the implementation type ✔️.

Of course, virtual member functions have issues. None bigger than the fact that they are intrusive. You simply cannot opt types that you do not own into an abstract interface, with the fundamental types not being able to opt into any abstract interface at all. And even when the intrusiveness isn't a total non-starter, we have issues with performance overhead and the need for allocation.

§ 1.2 Parametric Polymorphism

There's another interesting aspect of using virtual functions for polymorphism that's worth bringing up. Let's pick one of the more familiar generic interfaces in C++: `Iterator`. How would we implement `InputIterator` as an abstract base class?

```
struct InputIterator {
    // this one is fine
    virtual input_iterator& operator++() = 0;

    // this one is... questionable
    virtual bool operator==(input_iterator const&) const = 0;

    // .. but what about this one?
    virtual auto operator*() const -> ????;
};
```

We basically cannot make this as an interface. One problem is that we really don't want to make any two input iterators equality comparable to each other, regardless of what they iterate. But the even bigger problem is: what would `operator*` return here? There is no useful type we can put there that satisfies all input_iterators - we might want to return `int&` for some iterators, `std::string const&` for others, `double*` for others, etc.

What this example demonstrates is that `InputIterator` is a parameterized interface. And with virtual functions, the only we can provide those parameters is by adding template parameters. We take our interface and turn it into an interface template:

```
template <typename R,
          typename V = remove_cvref_t<R>,
          typename D = ptrdiff_t>
struct InputIterator {
    using value_type = V;
    using reference = R;
```

```
    using difference_type = D;

    // okay now we can do this one
    virtual reference operator*() const = 0;
};
```

But now we don't have an `InputIterator` interface. Not really, anyway. We have an `InputIterator<int&>` interface and an `InputIterator<std::string const&>` one. But that's not quite the idea we want to express. We call these additional parameters the *associated types* of an implementation.

Let's extend our list of requirements to include these, and present compliance in table form for easier reading:

	`virtual` member functions
Interface visible in code	✔️
Providing default implementations	✔️
Explicit opt-in	✔️
Diagnose incorrect opt-in	✔️
Easily invoke the customization	✔️
Verify implementation	✔️
Atomic grouping of functionality	✔️
Non-intrusive	❌
Associated Types	❌

1.3 Named Conformance vs Structural Conformance

One criteria in the above list is the ability to explicitly opt-in to interfaces. I actually consider this quite important.

There are two approaches to checking that a type meets an interface: structural conformance (validate that the signatures of an interface are satisfied) and named conformance (validate that the *name* of the interface is satisfied).

Virtual member function based polymorphism uses named conformance: you have to inherit, by name, of the interface you want to implement. C++ templates on the other hand, largely rely upon structural conformance. C++20 concepts as a language feature can only check structural conformance. However, sometimes structural checks are insufficient. There are already many cases in even just the standard library for *just* ranges in which the difference between two concepts cannot be expressed in a structural check and is purely semantic:

- `input_iterator` vs `forward_iterator`
- `range` vs `view`
- `range` vs `borrowed_range`
- `assignable` vs the checks that `indirectly_writable` does (arguably)

The way to express named conformance in is to use something like a type trait (what the first three of these do) or stick with a structural check that is just sufficiently weird as to not exist by accident (what the last one of these does).

A different, concrete example might be useful to demonstrate the necessary difference between named conformance and structural conformance. Let's say we wanted to create a customization point for erasing a given value from a container (as was added in [P1209R0]). We have the following very different interfaces:

```
// Erases all elements from 'container' that compare equal to 'value'
std::erase(container, value);

// Erase the element in 'container' pointed to by 'iterator'
container.erase(iterator);
```

Sure, for a given container, it's unlikely that we'd have some argument that *both* compares equal to its `value_type` *and also* is convertible to its `const_iterator`. But what happens *when* we come across such a case? Would we consider a container as opting into one interface when it's actually opting into the other? Or neither? Or if a container provides yet a different `erase` function that meets neither of these:

```cpp
template <typename T>
struct MyContainer {
    using iterator = /* ... */;
    using const_iterator = /* ... */;

    // erase by iterator, usual container interface
    iterator erase(iterator);
    iterator erase(const_iterator);

    // this container has to erase by index a lot, so
    // this is a convenient interface to avoid having to
    // write c.erase(c.begin() + idx) all the time
    iterator erase(ptrdiff_t idx) {
        return erase(begin() + idx);
    }
};
```

The author here may not have know about `std::erase(container, value)` and it would certainly be surprising to them (and other users) if `std::erase(container, 42)` on a `MyContainer<int>` instead of erasing those objects that have value `42` instead erased the object at index `42`.

The fact that we already even have this conflict in the standard library means that it's quite imperative to be vigilant with concept checks (and hopefully also demonstrates why any kind of unified function call syntax doesn't really help).

§ 2 Existing Static Polymorphism Strategies

C++ has two strategies for non-intrusive static polymorphism today:

1. Class Template Specialization
2. Free functions found by argument-dependent lookup (ADL), which can be subdivided further into:
    a. "pure" ADL
    b. customization point objects (see [N4381], 16.3.3.3.6 [customization.point.object])
    c. `tag_invoke` (see [P1895R0])

Not only are both of these non-intrusive, but neither have any additional runtime overhead, nor do either typically require allocation. But how well do they actually do at customization?

This paper will go through these four strategies in turn to see how well they apply to my criteria and where they succeed and where they come up wanting.

§ 2.1 Class Template Specialization

Class template specialization is less commonly used than ADL-based free functions, but it's certainly a viable strategy. Of the more prominent recent libraries, `fmt::format` ([fmtlib], now `std::format`) is based on the user specializing the class template `formatter` for their types. The format library is, without reservation, a great library. So let's see how well its main customization point demonstrates the facilities I describe as desirable for customization.

First, can we tell from the code what the interface is? If we look at the definition of the primary class template, we find:

```cpp
// A formatter for objects of type T.
template <typename T, typename Char = char, typename Enable = void>
struct formatter {
  // A deleted default constructor indicates a disabled formatter.
  formatter() = delete;
};
```

This tells us nothing at all ❌. You can certainly tell from this definition that is intended to be specialized by *somebody* (between the `Enable` template parameter and the fact that this class template is otherwise completely useless?) but you can't tell if it's intended to be specialized by the library author for the library's types or by the user for the user's types.

In this case, there is no “default” formatter - so it makes sense that the primary template doesn’t have any functionality. But the downside is, I have no idea what the functionality should be.

Now, yes, I probably have to read the docs anyway to understand the nuance of the library, but it’s still noteworthy that there is zero information in the code. This isn’t indicative of bad code either, the language facility doesn’t actually allow you to provide such.

The only real way to provide this information is with a concept. In this case, that concept could look like this. But the concept for this interface is actually fairly difficult to express (see 20.20.5.1 [formatter.requirements]).

Second, do we have the ability to provide default implementations that can be overridden? ❌ No, not really.

The `parse` function that the `formatter` needs to provide could have a meaningful default: allow only `"{}"` and parse it accordingly. But you can’t actually provide default implementations using class template specialization as a customization mechanism — you have to override *the whole thing*.

One way to (potentially) improve this is to separate `parse` and `format`. Maybe instead of a single `formatter` customization class, we have a `format_parser` for `parse` and `formatter` for `format`. At least, this is an improvement in the very narrow sense that the user could specialize the two separately – or only the latter. But I’m not sure it’s an improvement in the broader sense of the API of the library. It certainly seems much better to have a single customization entry for formatting, and all I’m describing here is a workaround for a language insufficiency. Alternatively, the formatting library could provide a class that you could inherit from that provides this default behavior. This means more work for the library author (providing each piece of default functionality as a separate component for convenient inheritance) and for the library consumer (that would need to explicitly inherit from each one).

Third, do we have the ability to opt in explicitly to the interface? ✔️ Yep! In fact, explicit opt in is the only way to go here. Indeed, one of the reasons some people dislike class template specialization as a mechanism for customization is precisely because to opt-in you have to do so outside of your class.

Fourth, is there any protection against implementing the interface incorrectly? ❌ Nope! There is nothing that stops me from specializing `formatter<MyType>` to behave like a `std::vector<MyType>`. There is no reason for me to actually do this, but the language supports it anyway. If you do it sufficiently wrong, it just won’t compile. Hopefully, the class author wrote a sufficiently good concept to verify that you implemented your specialization “well enough” so you get an understandable error message.

But worst case, your incorrect specialization coupled with insufficient vigilance and paranoia on the library author’s part might actually compile and just lead to bad behavior. What if your `std::hash` specialization accidentally returns `uint8_t` instead of `size_t`? What if you’re taking extra copies or forcing undesirable conversions? Took by reference instead of reference to const and are mutating? Very difficult to defend against this.

Fifth, can you easily invoke the customized implementation? ✔️ Yep! This isn’t really a problem with class template specialization. In this case, `formatter<T>::format` is the right function you want and is straightforward enough to spell. But you need to duplicate the type, which leads to potential problems. Do you get any protection against invoking the wrong implementation? ❌ Nope! You could call `formatter<U>::format` just as easily, and if the arguments happen to line up…?

The defense for this kind of error is that the customization point isn’t really user-facing, it’s only intended for internal consumption. In this case, used by `fmt::format` / `std::format`. This is best practice. But it’s something extra that needs to be provided by the class author. So I’ll give this one a 🤷 maybe.

Sixth, can you easily verify that a type implements an interface? Arguably, ❌ nope! Not directly at all. You can check that a specialization exists, but that doesn’t tell you anything about whether the specialization is correct. Compare this to the virtual function case, where checking if a `T*` is convertible to a `Base*` is sufficient for all virtual-function-based polymorphism.

Here, it would be up to the class author to write a `concept` that checks that the user did everything right. But this also something extra that needs to be provided by the class author.

Seventh, can we group multiple pieces of functionality atomically into one umbrella, such that failure to provide all of them can be diagnosed early? 🤷 Kind of. `formatter` is a good example here: while you cannot *only* provide a `parse` or *only* provide a `format` function (you *must* provide both), there isn’t anything in the language that enforces this. I can easily provide a specialization that only has one or the other (or neither), and this will only become an error at the point of use. In this sense, this is no different from any other incorrect implementation. But at least a missing customization point is much easier to diagnose than an incorrect one.

Eighth, is class template specialization non-intrusive? ✔️ Absolutely! Not much else to say here.

Ninth, does class template specialization support associated types? 🤷 Kind of. As with the common theme in this section, you *can* provide associated types in your specialization (indeed, what is `std::iterator_traits` if not a static polymorphism mechanism implemented with class template specialization whose entire job is to provide associated types?), there is nothing in the language that can *enforce* that these types exist. But, verifying the presence of type names (just like verifying the presence of functions) is a lot easier than verifying that a given function is properly implemented. Types are just easier, less to check.

So how'd we do overall? Let's update the table:

	`virtual` member functions	class template specialization
Interface visible in code	✔️	❌
Providing default implementations	✔️	❌
Explicit opt-in	✔️	✔️
Diagnose incorrect opt-in	✔️	❌
Easily invoke the customization	✔️	🤷
Verify implementation	✔️	❌
Atomic grouping of functionality	✔️	🤷
Non-intrusive	❌	✔️
Associated Types	❌	🤷

§ 2.2 Pure ADL-based customization

There has been innovation in this space over the years. We've used to have general guidelines about how to ensure the right thing happens. Then Ranges introduced to us Customization Point Objects. And now there is a discussion about a new model `tag_invoke`.

Ranges are probably the most familiar example of using ADL for customization points (after, I suppose, `<<` for iostreams, but as an operator, it's inherently less interesting). A type is a *range* if there is a `begin` function that returns some type `I` that models `input_or_output_iterator` and there is an `end` function that returns some type `S` that models `sentinel_for<I>`.

With pure ADL (ADL classic?), we would have code in a header somewhere (any of a dozen standard library headers brings it in) that looks like this:

```
namespace std {
    template <typename C>
    constexpr auto begin(C& c) -> decltype(c.begin()) {
        return c.begin();
    }

    template <typename T, size_t N>
    constexpr auto begin(T(&a)[N]) -> T* {
        return a;
    }

    template <typename C>
    constexpr auto end(C& c) -> decltype(c.end()) {
        return c.end();
    }

    template <typename T, size_t N>
    constexpr auto end(T(&a)[N]) -> T* {
        return a + N;
    }
}
```

Let's run through our criteria:

1. Can we see what the interface is in code? ❌ Nope! From the user's perspective, there's no difference between these function templates and anything else in the standard library.

2. Can you provide default implementations of functions? ✔️ Yep! The begin/end example here doesn't demonstrate this, but a different customization point would. `size(E)` can be defined as `end(E) - begin(E)` for all valid containers, while still allowing a

user to override it. Similarly, `std::swap` has a default implementation that works fine for most types (if potentially less efficient than could be for some). So this part is fine.

3. Can we opt in explicitly? ❌ Nope! You certainly have to explicitly provide `begin` and `end` overloads for your type to be a range, that much is true. But nowhere in your implementation of those functions is there any kind of annotation that you can provide that indicates *why* you are writing these functions. The opt-in is only implicit. For `begin`/`end`, sure, everybody knows what Ranges are — but for less universally known interfaces, some kind of indication of what you are doing could only help.

   On the other hand, you can certainly provide a function named `begin` for a type that has nothing to do with a range - it could be starting some task, or starting a timer, etc - and there's no way to say that this has nothing to do with ranges.

4. Is there protection against incorrect opt-in? ❌ Nope! What's stopping me from writing a `begin` for my type that returns `void`? Nothing. From the language's perspective, it's just another function (or function template) and those are certainly allowed to return `void`.

5. Can we easily invoke the customized implementation? ❌ Nope! Writing `begin(E)` doesn't work for a lot of containers, `std::begin(E)` doesn't work for others. A more dangerous example is `std::swap(E, F)`, which probably compiles and works fine for lots of times but is a subtle performance trap if the type provides a customized implementation and that customized implementation is not an overload in namespace `std`.

   Instead, you have to write `using std::swap; swap(E, F);` which while "easy" to write as far as code goes (in the sense that it's a formula that always works), I would not qualify as "easy" to always remember to do given that the wrong one works.

6. Can we easily verify the type implements an interface? ❌ I have to say no here. The "interface" doesn't even have a name in code, how would you check it? This isn't just me being pedantic - the only way to check this is to write a separate concept from the customization point. And this is kind of the point that I'm making - these are separate.

7. Does anything stop me from providing a non-member `begin` but not a non-member `end`? Nope ❌. This is similar to the class template specialization case: you can see at point of use that one or the other doesn't exist, but there's no way to diagnose this earlier.

8. Can we opt-in non-intrusively? ✔️ Yep! It's just as easy as writing a free function. No issues.

9. Can we add associated type support? ❌ I would say no. ADL is entirely about functions and not really about types. An associated type of the range concept would be it's iterator type, which is the type that `begin` returns. But it's not even easy to call that function, much less get its type properly. Would have to lean no here.

Not a great solution overall:

	`virtual` member functions	class template specialization	Pure ADL
Interface visible in code	✔️	❌	❌
Providing default implementations	✔️	❌	✔️
Explicit opt-in	✔️	✔️	❌
Diagnose incorrect opt-in	✔️	❌	❌
Easily invoke the customization	✔️	🤷	❌
Verify implementation	✔️	❌	❌
Atomic grouping of functionality	✔️	🤷	❌
Non-intrusive	❌	✔️	✔️
Associated Types	❌	🤷	❌

§ 2.3 Customization Point Objects

Customization Point Objects (CPOs) were designed to solve several of the above problems:

1. Provide an easy way to invoke the customized implementation. `ranges::swap(E, F)` just Does The Right Thing. ✔️.
2. Provide a way to to verify that a type implements the interface correctly, diagnosing some incorrect opt-ins. But it takes work. If a user provides a `begin` that returns `void`, `ranges::begin(E)` will fail at that point. This is not as early a failure as we get with

virtual member functions, but it's at least earlier than we would otherwise get. But I'm not really open to giving a full check, since the way `ranges::begin` does this verification is that the author of `ranges::begin` has to manually write it.

3. Provide a name for the interface that makes it easier to verify, which addresses the issue of interface verification. As above, it is possible to provide, but it must be done manually.

While `ranges::begin` and `ranges::end` do verify that those customization points properly return an iterator and a sentinel, and `ranges::range` as a concept verifies the whole interface, the fact that everything about this interface is implicit still leads to inherently and fundamentally poor diagnostics. Consider:

```
struct R {
    void begin();
    void end();
};
```

This type is not a `range`, obviously. But maybe I wanted it to be one and I didn't realize that `void` wasn't an iterator. What do compilers tell me when I try to `static_assert(std::ranges::range<R>)`?

msvc:

```
<source>(8): error C2607: static assertion failed
```

clang:

```
<source>:8:1: error: static_assert failed
static_assert(std::ranges::range<R>);
^                          ~~~~~~~~
<source>:8:28: note: because 'R' does not satisfy 'range'
static_assert(std::ranges::range<R>);
                           ^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-
        gnu/11.0.0/../../../../include/c++/11.0.0/bits/ranges_base.h:581:2: note: because 'ranges::begin(__t)'
        would be invalid: no matching function for call to object of type 'const __cust_access::_Begin'
        ranges::begin(__t);
        ^
```

gcc:

```
<source>:8:28: error: static assertion failed
    8 | static_assert(std::ranges::range<R>);
      |               ~~~~~~~~~~~~~^~~~~~~~
<source>:8:28: note: constraints not satisfied
In file included from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/string_view:44,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/basic_string.h:48,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/string:55,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/locale_classes.h:40,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ios_base.h:41,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/streambuf:41,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/streambuf_iterator.h:35,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/iterator:66,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/ranges:43,
                 from <source>:1:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:579:13:   required by the
        constraints of 'template<class _Tp> concept std::ranges::range'
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:579:21:   in requirements with
        '_Tp& __t' [with _Tp = R]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:581:22: note: the required
        expression 'std::ranges::__cust::begin(__t)' is invalid
  581 |         ranges::begin(__t);
      |         ~~~~~~~~~~~~~^~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:582:20: note: the required
        expression 'std::ranges::__cust::end(__t)' is invalid
  582 |         ranges::end(__t);
      |         ~~~~~~~~~~~^~~~~
cc1plus: note: set '-fconcepts-diagnostics-depth=' to at least 2 for more detail
```

If I crank up the diagnostics depth to 4 (2 is not enough), I finally get something about iterators in the 154 lines of diagnostic, reproduced here for clarity:

```
<source>:8:28: error: static assertion failed
    8 | static_assert(std::ranges::range<R>);
      |               ~~~~~~~~~~~~~^~~~~~~
<source>:8:28: note: constraints not satisfied
In file included from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/string_view:44,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/basic_string.h:48,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/string:55,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/locale_classes.h:40,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ios_base.h:41,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/streambuf:41,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/streambuf_iterator.h:35,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/iterator:66,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/ranges:43,
                 from <source>:1:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:579:13:   required by the
        constraints of 'template<class _Tp> concept std::ranges::range'
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:579:21:   in requirements with
        '_Tp& __t' [with _Tp = R]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:581:22: note: the required
        expression 'std::ranges::__cust::begin(__t)' is invalid, because
  581 |         ranges::begin(__t);
      |         ~~~~~~~~~~~~~^~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:581:22: error: no match for call
        to '(const std::ranges::__cust_access::_Begin) (R&)'
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:117:9: note: candidate:
        'template<class _Tp>  requires (__maybe_borrowed_range<_Tp>) && ((is_array_v<typename
        std::remove_reference<_Tp>::type>) || (__member_begin<_Tp>) || (__adl_begin<_Tp>)) constexpr auto
        std::ranges::__cust_access::_Begin::operator()(_Tp&&) const'
  117 |         operator()(_Tp&& __t) const noexcept(_S_noexcept<_Tp>())
      |         ^~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:117:9: note:   template argument
        deduction/substitution failed:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:117:9: note: constraints not
        satisfied
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h: In substitution of
        'template<class _Tp>  requires (__maybe_borrowed_range<_Tp>) && ((is_array_v<typename
        std::remove_reference<_Tp>::type>) || (__member_begin<_Tp>) || (__adl_begin<_Tp>)) constexpr auto
        std::ranges::__cust_access::_Begin::operator()(_Tp&&) const [with _Tp = R&]':
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:581:15:   required from here
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:117:2:   required by the
        constraints of 'template<class _Tp>  requires (__maybe_borrowed_range<_Tp>) && ((is_array_v<typename
        std::remove_reference<_Tp>::type>) || (__member_begin<_Tp>) || (__adl_begin<_Tp>)) constexpr auto
        std::ranges::__cust_access::_Begin::operator()(_Tp&&) const'
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:115:11: note: no operand of the
        disjunction is satisfied
  114 |         requires is_array_v<remove_reference_t<_Tp>> || __member_begin<_Tp>
      |                  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  115 |           || __adl_begin<_Tp>
      |           ^~~~~~~~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:114:18: note: the operand
        'is_array_v<std::remove_reference_t<_Tp> >' is unsatisfied because
  114 |         requires is_array_v<remove_reference_t<_Tp>> || __member_begin<_Tp>
      |                  ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  115 |           || __adl_begin<_Tp>
      |           ~~~~~~~~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:117:2:   required by the
        constraints of 'template<class _Tp>  requires (__maybe_borrowed_range<_Tp>) && ((is_array_v<typename
        std::remove_reference<_Tp>::type>) || (__member_begin<_Tp>) || (__adl_begin<_Tp>)) constexpr auto
        std::ranges::__cust_access::_Begin::operator()(_Tp&&) const'
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:114:18: note: the expression
        'is_array_v<typename std::remove_reference<_Tp>::type> [with _Tp = R&]' evaluated to 'false'
  114 |         requires is_array_v<remove_reference_t<_Tp>> || __member_begin<_Tp>
      |                  ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:114:57: note: the operand
        '__member_begin<_Tp>' is unsatisfied because
  114 |         requires is_array_v<remove_reference_t<_Tp>> || __member_begin<_Tp>
      |                  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~
  115 |           || __adl_begin<_Tp>
      |           ~~~~~~~~~~~~~~~~~~
In file included from /opt/compiler-explorer/gcc-trunk-
        20210102/include/c++/11.0.0/bits/stl_iterator_base_types.h:71,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/iterator:61,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/ranges:43,
```

```
                 from <source>:1:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:937:15:   required for the
        satisfaction of '__member_begin<_Tp>' [with _Tp = R&]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:937:32:   in requirements
        with '_Tp& __t' [with _Tp = R&]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:939:35: note: the required
        expression 'std::__detail::__decay_copy(__t.begin())' is invalid, because
  939 |           { __detail::__decay_copy(__t.begin()) } -> input_or_output_iterator;
      |             ~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:939:45: error: invalid use
        of void expression
  939 |           { __detail::__decay_copy(__t.begin()) } -> input_or_output_iterator;
      |                                    ~~~~~~~~~^~
In file included from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/string_view:44,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/basic_string.h:48,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/string:55,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/locale_classes.h:40,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ios_base.h:41,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/streambuf:41,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/streambuf_iterator.h:35,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/iterator:66,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/ranges:43,
                 from <source>:1:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:115:14: note: the operand
        '__adl_begin<_Tp>' is unsatisfied because
  114 |         requires is_array_v<remove_reference_t<_Tp>> || __member_begin<_Tp>
      |                  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  115 |           || __adl_begin<_Tp>
      |           ~~~^~~~~~~~~~~~~~~
In file included from /opt/compiler-explorer/gcc-trunk-
        20210102/include/c++/11.0.0/bits/stl_iterator_base_types.h:71,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/iterator:61,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/ranges:43,
                 from <source>:1:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:946:15:   required for the
        satisfaction of '__adl_begin<_Tp>' [with _Tp = R&]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:947:5:   in requirements
        with '_Tp& __t' [with _Tp = R&]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:949:35: note: the required
        expression 'std::__detail::__decay_copy(std::__detail::begin(__t))' is invalid, because
  949 |           { __detail::__decay_copy(begin(__t)) } -> input_or_output_iterator;
      |             ~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:949:41: error: use of
        deleted function 'void std::__detail::begin(auto:1&) [with auto:1 = R]'
  949 |           { __detail::__decay_copy(begin(__t)) } -> input_or_output_iterator;
      |                                    ~~~~~^~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/iterator_concepts.h:942:10: note: declared
        here
  942 |     void begin(auto&) = delete;
      |          ^~~~
In file included from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/string_view:44,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/basic_string.h:48,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/string:55,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/locale_classes.h:40,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ios_base.h:41,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/streambuf:41,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/streambuf_iterator.h:35,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/iterator:66,
                 from /opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/ranges:43,
                 from <source>:1:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:582:20: note: the required
        expression 'std::ranges::__cust::end(__t)' is invalid, because
  582 |         ranges::end(__t);
      |         ~~~~~~~~~~~^~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:582:20: error: no match for call
        to '(const std::ranges::__cust_access::_End) (R&)'
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:171:9: note: candidate:
        'template<class _Tp>  requires (__maybe_borrowed_range<_Tp>) && ((is_bounded_array_v<typename
        std::remove_reference<_Tp>::type>) || (__member_end<_Tp>) || (__adl_end<_Tp>)) constexpr auto
        std::ranges::__cust_access::_End::operator()(_Tp&&) const'
  171 |         operator()(_Tp&& __t) const noexcept(_S_noexcept<_Tp>())
      |         ^~~~~~~~
```

```
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:171:9: note:   template argument
       deduction/substitution failed:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:171:9: note: constraints not
       satisfied
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h: In substitution of
       'template<class _Tp>  requires (__maybe_borrowed_range<_Tp>) && ((is_bounded_array_v<typename
       std::remove_reference<_Tp>::type>) || (__member_end<_Tp>) || (__adl_end<_Tp>)) constexpr auto
       std::ranges::__cust_access::_End::operator()(_Tp&&) const [with _Tp = R&]':
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:582:13:   required from here
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:171:2:   required by the
       constraints of 'template<class _Tp>  requires (__maybe_borrowed_range<_Tp>) &&
       ((is_bounded_array_v<typename std::remove_reference<_Tp>::type>) || (__member_end<_Tp>) ||
       (__adl_end<_Tp>)) constexpr auto std::ranges::__cust_access::_End::operator()(_Tp&&) const'
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:169:9: note: no operand of the
       disjunction is satisfied
  168 |         requires is_bounded_array_v<remove_reference_t<_Tp>> || __member_end<_Tp>
      |                  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  169 |         || __adl_end<_Tp>
      |         ^~~~~~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:168:18: note: the operand
       'is_bounded_array_v<std::remove_reference_t<_Tp> >' is unsatisfied because
  168 |         requires is_bounded_array_v<remove_reference_t<_Tp>> || __member_end<_Tp>
      |                  ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  169 |         || __adl_end<_Tp>
      |         ~~~~~~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:171:2:   required by the
       constraints of 'template<class _Tp>  requires (__maybe_borrowed_range<_Tp>) &&
       ((is_bounded_array_v<typename std::remove_reference<_Tp>::type>) || (__member_end<_Tp>) ||
       (__adl_end<_Tp>)) constexpr auto std::ranges::__cust_access::_End::operator()(_Tp&&) const'
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:168:18: note: the expression
       'is_bounded_array_v<typename std::remove_reference<_Tp>::type> [with _Tp = R&]' evaluated to 'false'
  168 |         requires is_bounded_array_v<remove_reference_t<_Tp>> || __member_end<_Tp>
      |                  ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:168:65: note: the operand
       '__member_end<_Tp>' is unsatisfied because
  168 |         requires is_bounded_array_v<remove_reference_t<_Tp>> || __member_end<_Tp>
      |                  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~
  169 |         || __adl_end<_Tp>
      |         ~~~~~~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:134:15:   required for the
       satisfaction of '__member_end<_Tp>' [with _Tp = R&]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:134:30:   in requirements with
       '_Tp& __t' [with _Tp = R&]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:136:25: note: the required
       expression 'std::__detail::__decay_copy(__t.end())' is invalid, because
  136 |           { __decay_copy(__t.end()) }
      |             ~~~~~~~~~~~~^~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:136:33: error: invalid use of
       void expression
  136 |           { __decay_copy(__t.end()) }
      |                          ~~~~~~~^~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:169:12: note: the operand
       '__adl_end<_Tp>' is unsatisfied because
  168 |         requires is_bounded_array_v<remove_reference_t<_Tp>> || __member_end<_Tp>
      |                  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  169 |         || __adl_end<_Tp>
      |         ~~~^~~~~~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:144:15:   required for the
       satisfaction of '__adl_end<_Tp>' [with _Tp = R&]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:145:5:   in requirements with
       '_Tp& __t' [with _Tp = R&]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:147:25: note: the required
       expression 'std::__detail::__decay_copy(std::ranges::__cust_access::end(__t))' is invalid, because
  147 |           { __decay_copy(end(__t)) }
      |             ~~~~~~~~~~~~^~~~~~~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:147:29: error: use of deleted
       function 'void std::ranges::__cust_access::end(auto:3&) [with auto:3 = R]'
  147 |           { __decay_copy(end(__t)) }
      |                          ~~~^~~~
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/bits/ranges_base.h:140:10: note: declared here
  140 |     void end(auto&) = delete;
      |          ^~~
```

The issue here is explicitness. We have no idea if some random `begin` function we found is intended to be *the* entry point for `range` or not, so we don't know if a non-matching `begin` (whether the arguments don't line up or, as in this case, the return type doesn't meet requirements) is meaningful to diagnose or not.

This case might seem silly but it's actually very serious. Consider an example where instead of an "obvious" failure like trying to use `void` as an iterator, I actually had what I thought was a valid iterator, but was missing one operation or other (maybe it was missing postfix `operator++`, or its `operator*()` was not const-qualified?)? I'd get the exact same diagnostic: hundreds of lines of diagnostic, which simply *cannot* point to the problem.

It might seem that the single-line MSVC diagnostic of "static assertion failed" is something that reflects negatively on MSVC. But honestly, gcc's 154-line diagnostic when I crank up the diagnostic depth doesn't really provide me any meaningful information either.

All of which is to say, I'm only giving 🤷s to CPOs for verification.

Associated types are an interesting question for CPOs in their own right. Because it now becomes easy to invoke the right customization point, it also becomes easy to inspect those customization points. For instance, Ranges comes with type traits for the iterator and sentinel type of a range:

```
template<class T>
  using iterator_t = decltype(ranges::begin(declval<T&>()));
template<range R>
  using sentinel_t = decltype(ranges::end(declval<R&>()));
```

It's very convenient to have type traits to get these associated types, and these are highly important in ranges code. But it also means that we have a proliferation of type traits (Ranges alone has seven), which makes the API surface of the library absolutely enormous. So I'm going to give this a 🤷 as well.

Let's take a different interface. Let's say instead of Ranges and Iterators, we wanted to do equality. We'll have two functions: `eq` and `ne`. `eq` must be customized to take two `T const&`s and return `bool`. `ne` can be customized, but doesn't have to be, and defaults to negating the result of `eq`. As a CPO, this would look something like this (where my library is `N`):

```
namespace N::hidden {
    template <typename T>
    concept has_eq = requires (T const& v) {
        { eq(v, v) } -> std::same_as<bool>;
    };

    struct eq_fn {
        template <has_eq T>
        constexpr bool operator()(T const& x, T const& y) const {
            return eq(x, y);
        }
    };

    template <has_eq T>
    constexpr bool ne(T const& x, T const& y) {
        return not eq(x, y);
    }

    struct ne_fn {
        template <typename T>
            requires requires (T const& v) {
                { ne(v, v) } -> std::same_as<bool>;
            }
        constexpr bool operator()(T const& x, T const& y) const {
            return ne(x, y);
        }
    };
}

namespace N {
    inline namespace cpos {
        inline constexpr hidden::eq_fn eq{};
        inline constexpr hidden::ne_fn ne{};
    }

    template <typename T>
```

```
    concept equality_comparable =
        requires (std::remove_reference_t<T> const& t) {
            eq(t, t);
            ne(t, t);
        };
}
```

This is 42 lines of code.

It's worth reiterating that this is substantially better than raw ADL - if you just use `N::eq` and `N::ne` everywhere, you don't have to worry about issues like calling the wrong thing (perhaps some type has a more efficient inequality than simply negating equality? `N::ne` will do the right thing) or it being an invalid implementation (perhaps the user's implementation accidentally took references to non-const and mutated the arguments? This wouldn't compile). But this is not easy to write, and for such a straightforward interface, you can't really tell what it is anyway without some serious study. In this case, I didn't bother with the member/non-member rigamarole and only provided non-member opt-in. Providing member opt-in as well has some real cost in terms of both implementation complexity and diagnostics, so I'm sticking with the simple version for now.

CPOs improve upon just raw ADL names by allowing you to verify more things. While they provide the user a way to ensure they call the correct implementation and provide checking for the user that they implemented the customization point correctly (to some extent), that comes with a cost: somebody had to write all of that by hand, and it's not necessarily cheap to compile either. Even though we're addressing more of the customization facilities that I'm claiming we want, these are much harder and time-consuming interfaces to write.. that nevertheless are quite opaque.

	`virtual` member functions	class template specialization	Pure ADL	CPOs
Interface visible in code	✔️	❌	❌	❌
Providing default implementations	✔️	❌	✔️	✔️
Explicit opt-in	✔️	✔️	❌	❌
Diagnose incorrect opt-in	✔️	❌	❌	🤷
Easily invoke the customization	✔️	🤷	❌	✔️
Verify implementation	✔️	❌	❌	🤷
Atomic grouping of functionality	✔️	🤷	❌	❌
Non-intrusive	❌	✔️	✔️	✔️
Associated Types	❌	🤷	❌	🤷

2.4 `tag_invoke`

The `tag_invoke` paper ([P1895R0]) lays out two issues with Customization Point Objects (more broadly ADL-based customization points at large):

1. ADL requires globally reserving the identifier. You can't have two different libraries using `begin` as a customization point, really. Ranges claimed it decades ago.
2. ADL can't allow writing wrapper types that are transparent to customization.

This paper will discuss the second issue later. Instead I'll focus on the first point. This is an unequivocally real and serious issue. C++, unlike C, has namespaces, and we'd like to be able to take advantage that when it comes to customization. But ADL, very much by design, isn't bound by namespace. With virtual member functions, there are no issues with having `libA::Interface` and `libB::Interface` coexist (only if both provide virtual member functions of the same name and take the same parameters and a type wants to implement both). Likewise with class template specializations - specializing one name in one namespace has nothing to do with specializing a similarly-spelled name in a different namespace. But if `libA` and `libB` decide that they both want ADL customization points named `eq`? You better hope their arguments are sufficiently distinct or you simply cannot use both libraries.

The goal of `tag_invoke` is to instead globally reserve a single name: `tag_invoke`. Not likely to have been used much before the introduction of this paper.

The implementation of the `eq` interface introduced above in the `tag_invoke` model would look as follows:

```
namespace N {
    struct eq_fn {
```

```
        template <typename T>
            requires std::same_as<
                std::tag_invoke_result_t<eq_fn, T const&, T const&>, bool>
        constexpr bool operator()(T const& x, T const& y) const {
            return std::tag_invoke(*this, x, y);
        }
    };

    inline constexpr eq_fn eq{};

    struct ne_fn {
        template <typename T>
            requires std::invocable<eq_fn, T const&, T const&>
        friend constexpr bool tag_invoke(ne_fn, T const& x, T const& y) {
            return not eq(x, y);
        }

        template <typename T>
            requires std::same_as<
                std::tag_invoke_result_t<ne_fn, T const&, T const&>, bool>
        constexpr bool operator()(T const& x, T const& y) const {
            return std::tag_invoke(*this, x, y);
        }
    };

    inline constexpr ne_fn ne{};

    template <typename T>
    concept equality_comparable =
        requires (std::remove_reference_t<T> const& t) {
            eq(t, t);
            ne(t, t);
        };
}
```

This is 36 lines of code.

To what extent does this `tag_invoke`-based implementation of `eq` and `ne` address the customization facilities that regular CPOs fall short on? It does help: we can now explicitly opt into the interface (indeed, the only way to opt-in is explicit) ✔️!

But the above is harder to write for the library author (I am unconvinced by the claims that this is easier or simpler) and it is harder to understand the interface from looking at the code (before, the objects clearly invoked `eq` and `ne`, respectively, that is no longer the case). When users opt-in for their own types, the opt-in is improved by being explicit but takes some getting used to:

```
struct Widget {
  int i;

  // with CPO: just some function named eq
  constexpr friend bool eq(Widget a, Widget b) {
    return a.i == b.i;
  }

  // with tag_invoke: we are visibly opting
  // into support for N::eq
  constexpr friend bool tag_invoke(std::tag_t<N::eq>, Widget a, Widget b) {
    return a.i == b.i;
  }
};

// if we did this as a class template to specialize
template <>
struct N::Eq<Widget> {
    static constexpr bool eq(Widget a, Widget b) {
        return a.i == b.i;
    }

    // have no mechanism for providing a default
    // so it's either this or have some base class
    static constexpr bool ne(Widget a, Widget b) {
        return not eq(a, b);
```

```
    }
};
```

`tag_invoke` also doesn't really help on the diagnostics front. For this example, I'm requiring that `eq` return specifically `bool`. If I wanted to opt-in, and thus explicitly wrote a function named `tag_invoke`, but accidentally returned `int`? This is what I get from gcc:

```
<source>:64:18: error: static assertion failed
   64 | static_assert(N::equality_comparable<Widget>);
      |               ~~~^~~~~~~~~~~~~~~~~~~~~~~~~~~
<source>:64:18: note: constraints not satisfied
<source>:53:13:   required by the constraints of 'template<class T> concept N::equality_comparable'
<source>:54:9:   in requirements with 'std::remove_reference_t<_Tp>& t' [with T = Widget]
<source>:55:15: note: the required expression 'N::eq(t, t)' is invalid, because
   55 |             eq(t, t);
      |             ~~^~~~~~
<source>:55:15: error: no match for call to '(const N::eq_fn) (std::remove_reference_t<Widget>&,
        std::remove_reference_t<Widget>&)'
<source>:28:24: note: candidate: 'template<class T>  requires  same_as<typename
        std::invoke_result<xstd::tag_invoke_fn, N::eq_fn, const T&, const T&>::type, bool> constexpr bool
        N::eq_fn::operator()(const T&, const T&) const'
   28 |         constexpr bool operator()(T const& x, T const& y) const {
      |                        ^~~~~~~~
<source>:28:24: note:   template argument deduction/substitution failed:
<source>:28:24: note: constraints not satisfied
In file included from <source>:1:
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/concepts: In substitution of 'template<class T>
        requires  same_as<typename std::invoke_result<xstd::tag_invoke_fn, N::eq_fn, const T&, const T&>::type,
        bool> constexpr bool N::eq_fn::operator()(const T&, const T&) const [with T = Widget]':
<source>:55:15:   required from here
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/concepts:57:15:   required for the satisfaction of
        '__same_as<_Tp, _Up>' [with _Tp = int; _Up = bool]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/concepts:62:13:   required for the satisfaction of
        'same_as<typename std::invoke_result<xstd::tag_invoke_fn, N::eq_fn, const T&, const T&>::type, bool>'
        [with T = Widget]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/concepts:57:32: note: the expression
        'is_same_v<_Tp, _Up> [with _Tp = int; _Up = bool]' evaluated to 'false'
   57 |       concept __same_as = std::is_same_v<_Tp, _Up>;
      |                           ~~~~~^~~~~~~~~~~~~~~~~~~
<source>:56:15: note: the required expression 'N::ne(t, t)' is invalid, because
   56 |             ne(t, t);
      |             ~~^~~~~~
<source>:56:15: error: no match for call to '(const N::ne_fn) (std::remove_reference_t<Widget>&,
        std::remove_reference_t<Widget>&)'
<source>:45:24: note: candidate: 'template<class T>  requires  same_as<typename
        std::invoke_result<xstd::tag_invoke_fn, N::ne_fn, const T&, const T&>::type, bool> constexpr bool
        N::ne_fn::operator()(const T&, const T&) const'
   45 |         constexpr bool operator()(T const& x, T const& y) const {
      |                        ^~~~~~~~
<source>:45:24: note:   template argument deduction/substitution failed:
<source>:45:24: note: constraints not satisfied
<source>: In substitution of 'template<class T>  requires  same_as<typename
        std::invoke_result<xstd::tag_invoke_fn, N::ne_fn, const T&, const T&>::type, bool> constexpr bool
        N::ne_fn::operator()(const T&, const T&) const [with T = Widget]':
<source>:56:15:   required from here
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/concepts:57:15:   required for the satisfaction of
        '__same_as<_Tp, _Up>' [with _Tp = typename std::invoke_result<xstd::tag_invoke_fn, N::ne_fn, const T&,
        const T&>::type; _Up = bool]
/opt/compiler-explorer/gcc-trunk-20210102/include/c++/11.0.0/concepts:62:13:   required for the satisfaction of
        'same_as<typename std::invoke_result<xstd::tag_invoke_fn, N::ne_fn, const T&, const T&>::type, bool>'
        [with T = Widget]
<source>:56:15: error: no type named 'type' in 'struct std::invoke_result<xstd::tag_invoke_fn, N::ne_fn, const
        Widget&, const Widget&>'
   56 |             ne(t, t);
      |             ~~^~~~~~
```

There is *something* in the error message that says that `int` isn't the same type as `bool`. But it's not exactly easy to figure this out. Certainly as compared to a similar example involving virtual member functions:

```
<source>:65:9: error: conflicting return type specified for 'virtual int Widget::eq() const'
   65 |     int eq() const override { return 0; }
      |         ^~
<source>:61:18: note: overridden function is 'virtual bool Eq::eq() const'
```

```
   61 |     virtual bool eq() const = 0;
      |                 ^~
```

Let's add `tag_invoke` to the scoreboard:

	`virtual` member functions	class template specialization	Pure ADL	CPOs	`tag_invoke`
Interface visible in code	✔️	❌	❌	❌	❌
Providing default implementations	✔️	❌	✔️	✔️	✔️
Explicit opt-in	✔️	✔️	❌	❌	✔️
Diagnose incorrect opt-in	✔️	❌	❌	🤷	🤷
Easily invoke the customization	✔️	🤷	❌	✔️	✔️
Verify implementation	✔️	❌	❌	🤷	🤷
Atomic grouping of functionality	✔️	🤷	❌	❌	❌
Non-intrusive	❌	✔️	✔️	✔️	✔️
Associated Types	❌	🤷	❌	🤷	🤷

2.5 Better Enough?

If `tag_invoke` is improving on CPOs (and it is, even when I measure by criteria that are not related to the problems the authors set out to solve), why do I claim, as I do in the the title of this paper, that we need a language solution to this problem?

Because this is how I would implement the `eq`/`ne` interface in Rust (wherein this is called `PartialEq`):

```rust
trait PartialEq {
    fn eq(&self, rhs: &Self) -> bool;

    fn ne(&self, rhs: &Self) -> bool {
        !self.eq(rhs)
    }
}
```

This is 7 lines of code, even including the empty line and the two lines containing a single close brace. This trivial implementation, which you probably understand even if you don't know Rust, *easily* meets all of the criteria presented thus far. And unlike CPOs and `tag_invoke`, where the extent of the ability to protect the user from faulty implementations or provide them with interface checks is dependent on the class author writing them correctly, here these checks are handled by and provided by the language. As a result, the checks are more robust, and the interface author doesn't have to do anything.

Moreover, it even meets one of `tag_invoke`'s stated criteria: it does not globally reserve names. Though it does not meet the other: you cannot transparently implement and pass-through a trait that you do not know about.

Ultimately, I want us to aspire to more than replacing one set of library machinery that solves a subset of the problem with a different set of library machinery that solves a larger subset of the problem… where neither set of library machinery actually gives you insight into what the interface is to begin with.

To make this more clear:

	`virtual` member functions	class template specialization	Pure ADL	CPOs	`tag_invoke`	Rust Traits
Interface visible in code	✔️	❌	❌	❌	❌	✔️
Providing default implementations	✔️	❌	✔️	✔️	✔️	✔️
Explicit opt-in	✔️	✔️	❌	❌	✔️	✔️
Diagnose incorrect opt-in	✔️	❌	❌	🤷	🤷	✔️
Easily invoke the customization	✔️	🤷	❌	✔️	✔️	✔️
Verify implementation	✔️	❌	❌	🤷	🤷	✔️
Atomic grouping of functionality	✔️	🤷	❌	❌	❌	✔️
Non-intrusive	❌	✔️	✔️	✔️	✔️	✔️
Associated Types	❌	🤷	❌	🤷	🤷	✔️

2.6 The swap example

To help drive home the significance of the diagnostics and the fact that neither CPOs nor `tag_invoke` can possibly provide them, consider the following incorrect opt-in to `swap`:

```
namespace N {
    template <typename T>
    struct Widget { ... };

    template <typename T>
    void swap(Widget<T>&, Widget<T> const&);
}
```

Regardless of if we're using CPOs or `tag_invoke`, what we have here is an incorrect opt-in to `swap`. We need a function that takes two `T&`s but we accidentally made one of them `const`. This error is not diagnosable.

Why not? Both implementations basically detect the customization point by seeing if they can find a valid candidate for `swap(x, x)` for a `T& x`. There's no mechanism we have to detect the difference between a `swap` that exists and is wrong (like the above) and a `swap` that doesn't exist. We just detect that `swap(x, x)` is an invalid expression. The result is that we fall-back to using the default implementation of `swap`, and get no hint that we did something wrong! The only way we'd notice this if actually detected a slow-down of some kind, or were specifically testing the `swap` here very carefully.

A different kind of incorrect opt-in would be if we'd accidentally done this:

```
template <typename T>
void swap(Widget<T>&, Widget<T>);
```

That is, take the second parameter by value instead of by lvalue-reference. Here, our candidate *would* get selected, but just not actually do a proper swap. We'd eventually discover this error by seeing `swap` fail to actually `swap` one of the arguments.

A third kind of incorrect opt-in would be if we put it in the wrong namespace:

```
namespace N {
    namespace Inner {
        template <typename T>
        struct Widget { ... };
    }

    template <typename T>
    void swap(Inner::Widget<T>&, Inner::Widget<T>&);
}
```

Here, we finally got the parameters right. But ADL won't find this overload, since `N` isn't an associated namespace of `N::Inner::Widget<T>`, only `N::Inner` is.

The first problem is something that might be guarded against, perhaps by verifying that `swap(c, c)` for a `T const& c` does not find a candidate if `swap(x, x)` did not, or some other similar implementation heroics. But this basically means doing an extra bout of overload resolution for every swap, even when *not* providing a custom swap is fairly typical, so seems unlikely to be done. I'm not sure how you could guard against either the second or third problems.

The problem here is we're not just writing a function template whose name is `swap` - we're very specifically opting into an interface. It's just that unlike virtual member functions, we have no way of expressing this intent today. And without that intent, we can't get diagnostics for such mistakes.

3 Relevant Work

I don't want to just point to Rust and ask that we keep up. I also want to highlight existing work in C++ specifically that can address this problem as well.

3.1 Customization Point Functions

One paper that addresses this topic is Matt Calabrese's [P1292R0]. This paper proposes a language facility that is a direct translation of C++ virtual member functions from the dynamic polymorphism realm into the static polymorphism realm. We can implement the

recurring example in this paper with such a facility as follows:

```
namespace N {
    template <typename T>
    virtual constexpr auto eq(T const&, T const&) -> bool = 0;

    template <typename T>
    virtual constexpr auto ne(T const& x, T const& y) -> bool {
        return not eq(x, y);
    }

    template <typename T>
    concept equality_comparable =
        requires (std::remove_reference_t<T> const& t) {
            eq(t, t);
            ne(t, t);
        };
}
```

Which is now just 16 lines of code.

We would opt-in to this facility by providing an `override`:

```
struct Widget {
    int i;
};

auto eq(Widget const& x, Widget const& y) -> bool override : N::eq {
    return x.i == y.i;
}
```

This is a far, far simpler implementation for the library author, that is easier to understand for the reader, and does a lot more for us, since the language can do more checking for us. It's definitely a big step between `tag_invoke` and Rust:

	`virtual` member functions	class template specialization	Pure ADL	CPOs	`tag_invoke`	customization point functions	Rust Traits
Interface visible in code	✔️	❌	❌	❌	❌	✔️	✔️
Providing default implementations	✔️	❌	✔️	✔️	✔️	✔️	✔️
Explicit opt-in	✔️	✔️	❌	❌	✔️	✔️	✔️
Diagnose incorrect opt-in	✔️	❌	❌	🤷	🤷	✔️	✔️
Easily invoke the customization	✔️	🤷	❌	✔️	✔️	✔️	✔️
Verify implementation	✔️	❌	❌	🤷	🤷	🤷	✔️
Atomic grouping of functionality	✔️	🤷	❌	❌	❌	❌	✔️
Non-intrusive	❌	✔️	✔️	✔️	✔️	✔️	✔️
Associated Types	❌	🤷	❌	🤷	🤷	🤷	✔️

While customization point functions have several clear benefits, they still don't address all the issues here. In particular, when dealing with an interface that logically has multiple customization points, there's no way of aggregating them together (short of providing a concept that has to unify them), and so there's nothing to prevent a user from doing something like providing an override for `ne` but not for `eq`.

And customization point functions are limited to the kinds of things that functions can do, so they likewise don't provide any added benefit where associated types are concerned as compared to CPOs or `tag_invoke`. They still require type traits for everything interesting.

What customization point functions *do* provide is an ability to potentially address the other issue `tag_invoke` sought to solve: the ability to forward customizations. With P1292, we already have a dissociation between the *name* of the override and the *name* of the function that it is overriding. The paper provides the following example:

```
template <class It, class Distance>
virtual constexpr void advance(It& it, Distance n)
    requires InputIterator<It>
{
    for (; n != 0; --n) {
        ++it;
    }
}

template <class It, class Distance>
constexpr void advance_bidirectional(It& it, Distance n) override
  requires BidirectionalIterator<It>
  : advance
{
    if (n >= 0) {
        for (; n != 0; --n) {
            ++it;
        }
    } else {
        for(; n != 0; ++n) {
            --it;
        }
    }
}
```

Lewis Baker (one of the `tag_invoke` authors) suggests an extension to this direction that allows deducing the the customization point being overriden. As in (the following example is reduced somewhat from a similar one in [P2175R0], and takes the liberty of assuming we can implement customization point functions as members — an idea which does not appear in Matt's paper at all):

```
template <typename Receiver>
struct receiver {
    Receiver inner;

    // Override get_stop_token()
    auto get_stop_token() const -> std::never_stop_token
            override : std::execution::get_stop_token
    {
        return {}
    }

    // Pass through other customization points
    template <auto CPO, typename Self, typename... Args>
    auto fwd_cpo(this Self&& self, Args&&... args) -> decltype(CPO(FWD(self).inner, FWD(args)...))
            override: CPO
    {
        return CPO(FWD(self).inner, FWD(args)...);
    }
};
```

Definitely something to seriously consider. One issue might be how to figure out how to pick the right overrides. But collecting overrides and relying on them to be constrained seems likely to produce a smaller set of candidates than having to perform name lookup across all associated namespaces and classes.

§ 3.2 Reflective Metaprogramming

I've pointed out a few times the relative sizes of the solutions presented thus far: that the CPO solution requires 42 lines of code and the `tag_invoke` solution requires 36, customization point functions allow us to reduce this to 16, while the Rust traits example only requires 7. One follow-up question to this is: to what extent can reflective (generative) metaprogramming address this need?

Consider a block of code like the following (I'm using the stereotypes suggested in [P2237R0]. The particular syntax chosen here might be incorrect, but probably isn't relevant to the point I'm trying to make):

```
namespace N {
    template <typename T>
    <<virtual_>> constexpr auto eq(T const&, T const&) -> bool;

    template <typename T>
        requires requires (T const& x, T const& y){
            eq(x, y);
        }
    <<virtual_>> constexpr auto ne(T const& x, T const& y) -> bool {
        return not eq(x, y);
    }
}
```

This looks quite a bit like the customization point function implementation, right? Let's see what we could do with such a thing. We could implement `virtual_` to produce class templates that look like this:

```
namespace N::virtual_ {
    template <typename T>
    struct eq_t;

    template <typename T>
    using eq_parameters = mp_list<T const&, T const&>;

    template <typename T>
    concept eq_return = std::same_as<T, bool>;
}

namespace N {
    inline constexpr auto eq =
        []<typename T>
            requires requires (virtual_::eq_t<T> f, T const& x, T const& y) {
                { f(x, y) } -> virtual_::eq_return;
            }
        (T const& x, T const& y) -> bool {
            return virtual_::eq_t<T>{}(x, y);
        };
}

namespace N::virtual_ {
    template <typename T>
    struct ne_t;

    template <typename T>
        requires requires (T const& x, T const& y){
            eq(x, y);
        }
    struct ne_t<T> {
        constexpr auto operator()(T const& x, T const& y) const -> bool {
            return not eq(x, y);
        }
    };

    template <typename T>
    using ne_parameters = mp_list<T const&, T const&>;

    template <typename T>
    concept ne_return = std::same_as<T, bool>;
}

namespace N {
    inline constexpr auto ne =
        []<typename T>
            requires requires (virtual_::ne_t<T> f, T const& x, T const& y) {
                { f(x, y) } -> virtual_::ne_return;
            }
        (T const& x, T const& y) -> bool {
            return virtual_::ne_t<T>{}(x, y);
        };
}
```

The algorithm here would be that for each function template `F` annotated by `<<virtual_>>`:

1. Introduce a class template `F_t` into `N::virtual_` with the same template parameters as `F`. If `F` has no definition, the class template has no definition (as in `eq`). If `F` has a definition, then copy it as the call operator. If `F` has a definition with constraints (as in `ne`), then introduce an empty primary class template and additionally add a constrained specialization, copying the constraints.

2. Introduce an alias template that stashes the types of the parameters in `F` for a given instantiation. This is probably not the right way to do this, but we do need to store *some* metadata somewhere about what the parameters need to be for this function in a way to allow us to verify them later. We also introduce a concept for the return type. If `F` returns a type `T` that is not `void`, then that concept is `same_as<T>`. If `F` returns `C auto`, then that constraint is `C`. Otherwise, the constraint is just `true`.

3. Introduce a lambda into `N` that is a transformed version of `F` that invokes `N::virtual_::F_t` when that is a valid expression whose return type satisfies `N::virtual_::F_return`.

Rather than doing CPOs or `tag_invoke`, I'm actually going back to class template specialization here. But I'm trying to use reflection to hide all the problems with it, but keep its benefits (notably, a much smaller lookup space). But if the class template is hidden, how would you opt-in? We continue the stereotype approach and provide `override` stereotype akin to the `override` facility presented in the customization point functions paper.

As in:

```
namespace N {
    template <typename T> struct Widget { ... };

    template <typename T>
    <<override(std::ranges::swap)>> void swap(Widget<T>& x, Widget<T> const& y) { ... }
}
```

Such a stereotype could do all of the following:

1. Verify that `std::ranges::swap` is indeed a `virtual_` customization point. It is not at the moment (and this could be diagnosed at the point of declaration here), but let's pretend it is for the sake of argument.

2. Verify that this implementation matches the interface of the `std::ranges::swap` customization point, by directly examining the function parameters and the return type this would let us reject the bad opt-in of `swap` at its point of declaration — for any of the incorrect opt-ins to swap presented earlier. In this case (based on my guess implementation earlier), the parameters of swap would be `mp_list<T&, T&>`, so we should be able to reject the list `mp_list<Widget<T>&, Widget<T> const&>` as being incompatible. Alternatively, we store the whole function signature and verify that it's more specialized than the original. I may not be 100% sure what the right way to check this might be, but it at least seems to be that it's *possible* to implement `virtual_` and `override` such that we *could* check this.

3. Once we do both of those verifications, rewrite the provided definition to be a function template specialization (which is more difficult because we both have to figure out how to perform the specialization and would need to allow ourselves to specialize class templates in a different namespace that doesn't change lookup context?):

    ```
    template <typename T>
    struct ::std::ranges::virtual_::swap_t<Widget<T>> {
        void operator()(Widget<T>& x, Widget<T>& y) const { ... }
    };
    ```

Which should allow a library implementation to now accurately diagnose incorrect opt-ins, while providing an explicit opt-in syntax very similar to [P1292R0].

Arguably the constraints that I illustrated earlier on the function objects aren't actually necessary. That is, I showed `eq` as:

```
namespace N {
    inline constexpr auto eq =
        []<typename T>
            requires requires (virtual_::eq_t<T> f, T const& x, T const& y) {
                { f(x, y) } -> virtual_::eq_return;
            }
        (T const& x, T const& y) -> bool {
            return virtual_::eq_t<T>{}(x, y);
        };
}
```

We don't need to verify the expression `f(x, y)`. The `override` stereotype already does that for us. We just need to validate that `virtual_::eq_t<T>` is a valid type, which should reduce some compile overhead:

```
namespace N {
    inline constexpr auto eq =
        []<typename T>
            requires requires {
                virtual_::eq_t<T>{};
            }
        (T const& x, T const& y) -> bool {
            return virtual_::eq_t<T>{}(x, y);
        };
}
```

The above requires a lot of new language features, but if what I'm describing here is actually implementable (and there are certainly *many* questions here about that), then we may be able to implement customization point functions exactly as a library. With all of their benefits (as compared to `tag_invoke`: having the implementation visible in code and the ability to diagnose incorrect opt-ins) and their weaknesses (no way of grouping multiple customization points into a cohesive unit, providing an easy verification for that grouping, or support for associated types).

I also haven't the slighest idea how to do forwarding of arbitrary customization points in this model.

An important downside to this approach as compared to the customization point functions language feature is prvalue propagation. With `virtual` member functions and the `virtual` "free" functions design, if I have a `virtual` function that takes a prvalue, invoking the customization point *directly* invokes the most derived implementation with a prvalue. That is, the prvalue is materialized at its target. The same is true of "pure" ADL-based customization points, since we just invoke the target function.

But this is not the case with CPOs, `tag_invoke`, or the above reflection-based implementation of class template specializations. In each of these cases, we invoke a function object that dispatches to the most derived implementation. This means the prvalue must be materialized earlier and then moved. This is a known gotcha with implementing something like `std::function<void(std::string)>` — it can't quite be as good as you'd want it to be, because you end up with *two* functions in your call chain taking a `std::string` (or, if you implement it poorly, more than two).

Perhaps there's yet another language feature that could facilitate efficient prvalue materialization here? Expression aliases? Lazy parameters?

§ 3.3 C++0x Concepts

Rust is hardly the only language that can solve this problem. Indeed, C++0x Concepts [N1758] gave us a solution that is nearly identical to the Rust one (this appears in the paper under the name `EqualityComparable`, I'm just changing it to match the names used throughout the paper):

Rust	C++0x

`trait PartialEq {`
`  fn eq(&self, rhs: &Self) -> bool;`
 	
`  fn ne(&self, rhs: &Self) -> bool {`
`  !self.eq(rhs)`
`  }`
`}`
	`template <typeid T>`
	`concept PartialEq {`
	`  auto eq(T const&, T const&) -> bool;`
	
	`  auto ne(T const& x, T const& y) -> bool {`
	`  return not eq(x, y);`
	`  }`
	`};`

The differences here are completely aesthetic; this solution performs just as well as the Rust one. Were I to be consistent with the other examples and stash this in `namespace N`, this would be just 10 lines of code (compared to 16 with customization point functions, 36 with `tag_invoke`, and 42 with customization point objects).

	`virtual` member functions	class template specialization	Pure ADL	CPOs	`tag_invoke`	customization point functions	Rust Traits	C++0x Concepts
Interface visible in code	✔	✘	✘	✘	✘	✔	✔	✔

	`virtual` member functions	class template specialization	Pure ADL	CPOs	`tag_invoke`	customization point functions	Rust Traits	C++0x Concepts
Providing default implementations	✔️	❌	✔️	✔️	✔️	✔️	✔️	✔️
Explicit opt-in	✔️	✔️	❌	❌	✔️	✔️	✔️	✔️
Diagnose incorrect opt-in	✔️	❌	❌	🤷	🤷	✔️	✔️	✔️
Easily invoke the customization	✔️	🤷	❌	✔️	✔️	✔️	✔️	✔️
Verify implementation	✔️	❌	❌	🤷	🤷	🤷	✔️	✔️
Atomic grouping of functionality	✔️	🤷	❌	❌	❌	❌	✔️	✔️
Non-intrusive	❌	✔️	✔️	✔️	✔️	✔️	✔️	✔️
Associated Types	❌	🤷	❌	🤷	🤷	🤷	✔️	✔️

What we saw in each example so far - with customization point objects, with `tag_invoke`, and with customization point functions - was that we have to take these independent customization points and group them into `concept` into order to indicate that they are closely related.

What C++0x Concepts showed us was that we could simply start from the grouped collection of customization points instead. But the opt-in mechanism for C++0x concepts was a little different: we have concept maps (see [N2042]). The `Widget` opt-in from earlier would be:

```
struct Widget { int i; };

template <>
concept_map PartialEq<Widget> {
    auto eq(Widget const& x, Widget const& y) -> bool {
        return x.i == y.i;
    }
};
```

The invocation model is quite different too. Using customization point functions, `N::eq` is just a function that I can invoke wherever. Indeed it also behaves as an object, so I can pass it as an algorithm to a different algorithm (e.g. `views::group_by(N::eq)` is perfectly valid). But C++0x Concepts didn't have this idea that `PartialEq::eq` would be any kind of callable. Which makes it entirely non-obvious to figure out how to forward a customization point.

```
template <typename Receiver>
struct receiver {
    Receiver inner;
};

// for a concrete concept, fine
template <typename R>
concept_map GetStopToken<receiver<R>> {
    auto get_stop_token(receiver<R> const&) const -> std::never_stop_token {
        return {};
    }
};

// for an arbitrary one?? Well, whatever concept this is, C, needs to be satisfies by R
template <template <typename> concept C, C R>
concept_map C<receiver<R>>
{
    // but what in the world do we put here???
    template <typename Self, typename... Args>
```

```
    auto ???(this Self&& r, Args&&... args) -> decltype(auto) {
        return ???(FWD(r).inner, FWD(args)...);
    }
}
```

Customization point functions give us an aswer - since the customization point function itself is an object that gives us some nice properties. But in this concepts model, not so much.

3.4 The contenders

Let's append customization forwarding to our table and drop all the other options I've discussed thus far, save for three:

	`tag_invoke`	customization point functions	C++0x Concepts
Interface visible in code	❌	✔️	✔️
Providing default implementations	✔️	✔️	✔️
Explicit opt-in	✔️	✔️	✔️
Diagnose incorrect opt-in	🤷	✔️	✔️
Easily invoke the customization	✔️	✔️	✔️
Verify implementation	🤷	🤷	✔️
Atomic grouping of functionality	❌	❌	✔️
Non-intrusive	✔️	✔️	✔️
Associated Types	🤷	🤷	✔️
Forwarding Customizations	✔️	✔️	❌

I'm giving customization point functions credit for customization forwarding, even though that paper makes no mention of such a thing, since at least I'm under the impression that it's a direction that could be pursued.

`tag_invoke` is an improvement over customization point objects as a library solution to the static polymorphism problem. But I don't really think it's better enough, and we really need a language solution to this problem. I'm hoping this paper is a good starting point for a discussion, at least.

4 References

[fmtlib] Victor Zverovich. 2012. fmtlib.
https://fmt.dev/latest/index.html

[N1758] J. Siek, D. Gregor et al. 2005-01-17. Concepts for C++0x.
https://wg21.link/n1758

[N2042] D. Gregor, B. Stroustrup. 2006-06-24. Concepts.
https://wg21.link/n2042

[N4381] Eric Niebler. 2015-03-11. Suggested Design for Customization Points.
https://wg21.link/n4381

[P1209R0] Alisdair Meredith, Stephan T. Lavavej. 2018-10-04. Adopt Consistent Container Erasure from Library Fundamentals 2 for C++20.
https://wg21.link/p1209r0

[P1292R0] Matt Calabrese. 2018-10-08. Customization Point Functions.
https://wg21.link/p1292r0

[P1895R0] Lewis Baker, Eric Niebler, Kirk Shoop. 2019-10-08. tag_invoke: A general pattern for supporting customisable functions.
https://wg21.link/p1895r0

[P2175R0] Lewis Baker. 2020-12-15. Composable cancellation for sender-based async operations.
https://wg21.link/p2175r0

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
https://wg21.link/p2237r0